UI Audit

Recipe App

Date	28 July 2026
Auditor	Claude Opus 5 — automated walkthrough
Environment	Local dev (Vite :5173 + mock API :3001)
Database	Neon "ui-audit" branch — isolated copy; production untouched
Coverage	15 routes · 6 widths incl. iPhone 16 Pro · 7 workflows

Severity	Count	
P0	0	broken / blocking
P1	5	data loss / misleading / task obstructed
P2	21	confusing / friction
P3	6	polish

Layout, contrast and typography are in good shape. The findings concentrate in destructive actions without confirmation, controls that disable or no-op silently, and inconsistent semantic structure.

Date: 2026-07-28 **Auditor:** Claude Opus 5 (automated walkthrough) **Environment:** local Vite dev server (`localhost:5173`) + mock API (`localhost:3001`), Neon `ui-audit` branch (isolated copy of production — production untouched) **Account:** `JordanTest2` **Routes covered:** 15 · **Widths tested:** 375 / 525 / 627 / 1280 / 1920 · **Workflows walked:** 7

Method (read this first)

The audit ran in two passes.

Pass 1 — measurement, without screenshots. The browser pane was not being displayed, so it was not compositing frames and every `computer{action:"screenshot"}` call failed. Rather than stop, the audit was run against the accessibility tree, page text, and injected scripts computing real values from the live DOM: contrast ratios (WCAG formula with sRGB and alpha compositing), element geometry for touch targets, horizontal-overflow detection, heading order, and label association.

Pass 2 — visual review. Once the pane was displayed, screenshots worked and a visual pass was run over the core screens at 375px. This pass found **six additional findings that measurement alone had missed** (F-024 – F-029), including the most serious layout defect in the report (F-024). Viewport resizing also became reliable at that point — the same root cause explains why Pass 1's resize requests were ignored (`resize_window` reporting 375px while `innerWidth` still read 627px).

Two methodology notes worth keeping:

- The first contrast pass produced **false positives**: this app uses modern `color(srgb …)` values in the 0–1 range,

which a naive parser reads as 0–255. The parser was corrected and every contrast number below was re-measured. The initially-suspected "Add new / Discover" failure was **disproved** (it measures 5.38:1 and passes).

- The long-title stress test (F-020) was measured against a synthetic 343px container, not a live phone viewport.

Still not covered: a full screenshot-per-finding evidence set (the browser tooling returns images to the reviewer but cannot write them to disk, so the PDF describes visual findings rather than embedding them), the complete ten-width breakpoint matrix, and the areas listed under *Skipped / untested* at the end.

Executive summary

Severity	Count
P0 — broken / blocking	0
P1 — data loss, actively misleading, or core task obstructed	5
P2 — confusing / friction	21
P3 — polish	6

Six findings were fixed and verified on 28 Jul 2026 — F-007, F-010, F-011, F-012, F-016, F-020. Each is marked inline below. The remaining items are untouched, and the ones needing a product decision are listed under *Deliberately not fixed* at the end.

The headline: the app's *layout* is in good shape — no horizontal overflow at any width tested, contrast is essentially clean, typography is genuinely well-executed (74-character lines, 1.45 line-height, content capped on wide screens), and the responsive behaviour holds from 375px to 1920px. The problems are concentrated in **destructive actions and feedback**, not in visual design.

Three systemic themes drive most findings:

1. **Destructive actions have no confirmation and no undo.** Two separate one-click paths wipe the shopping list.
2. **Controls disable or no-op silently.** Three primary buttons grey out with the reason available only to screen readers; the servings stepper does nothing at all for 25% of recipes while still counting up.
3. **Semantic structure is inconsistent with an otherwise careful ARIA implementation.** The app has excellent `aria-labels` on interactive controls but is missing `h1` on 6 of 15 routes, and error messaging is announced in one place (toasts) and silent in another (sign-in).

P1 findings

F-001 · "Clear list" destroys the entire shopping list with no confirmation and no undo

Route: `/#/shopping` · **Category:** data-loss

One click on **Clear list** removed every recipe, every ingredient, all purchased/checked-off state, and all manually added items. No confirmation dialog appeared (verified by hooking `window.confirm` — it was never called), no `role="dialog"` was rendered, and no undo was offered.

Repro: build a list → check items off → add a manual item → click "Clear list". Everything is gone instantly.

Why it matters: this is the highest-cost mistake in the app. A shopping list is built up over a week and checked off *in a store* — losing it mid-shop is unrecoverable.

Fix: confirmation dialog naming what will be lost ("Clear 14 items, including 3 you've checked off?"), or an undo toast. The app already has a toast system that could carry the undo.

F-002 · "Shop ingredients" silently replaces the existing shopping list

Route: `/#/` → `/#/shopping` · **Category:** data-loss

Using **Shop ingredients** from the menu rebuilds the shopping list from the current selection and **discards the previous list's recipes and their checked-off state**, with no warning. Manually added items survive; recipe-derived items do not.

Repro: shop for recipes A + B → return to menu → select only recipe B → "Shop ingredients". Recipe A and all its ingredients are gone, silently.

The button *does* carry the warning — but only in its `aria-label`: `"Shop ingredients — select meals on this week's menu (replaces your shopping list when used)"`. A sighted user never sees this; it is invisible outside a screen reader or a tooltip hover.

Fix: surface "replaces your current list" as visible text next to the button, and confirm when the existing list is non-empty.

F-003 · Any API failure blanks the entire app with a developer error string

Route: all · **Category:** error-handling

With the API unavailable, the whole application renders exactly one line of text:

```
Failed to load /api/ingredients (500)
```

No header, no navigation, no retry control, no human-readable explanation — and it leaks an internal endpoint path and HTTP status to the user. The only recovery is knowing to reload the page. (Verified by stopping `local-api.mjs`, reloading, then restarting it — the app recovered on reload.)

Why it matters: this is the failure mode most likely to be hit in the real world (flaky network, cold start, deploy). A transient blip currently looks like a totally broken app.

Fix: render the error inside the normal app chrome, with plain language ("Couldn't load your ingredients") and a **Retry** button. Keep the technical detail in the console.

F-004 · Servings stepper is a silent no-op for 25% of recipes

Routes: `/#/shopping`, cook mode · **Category:** correctness / misleading

For recipes with no base servings recorded, changing servings updates the displayed number but **does not change a single ingredient quantity** — with no indication anything is wrong.

Measured: for *Beef tortilla melts2*, servings 1 → 4 left every quantity identical (Ground beef 10 oz, Cheddar 0.50 cup, Sour cream 3 tbsp unchanged at every step).

Confirmed working where the data exists: *Chicken Parmesan* (base 4) at 4 → 8 servings correctly doubled — chicken breast 1.50 lb → 3 lbs, panko 1 cup → 2 cups, with correct pluralisation.

Root cause is in the code, not the arithmetic — `RecipeCookModePanel.tsx:235`: `const ingredientScale = baseServings ? cookServings / baseServings : 1;` — falling back to `1` whenever base servings are missing. `ShoppingListPage.tsx:167` does the same.

Scope, from the database: 18 of 71 recipes (25%) have `servings IS NULL`.

Why it matters: the user scales to 8 servings, shops from that list, and buys half the food they need. The UI gives no hint that the control is inert.

Fix: when base servings are missing, either disable the stepper with a visible "set servings on this recipe first" link, or prompt for base servings on first scale. Backfilling the 18 recipes is a data fix, not a UI fix — the UI should still handle the null case honestly.

P2 findings

F-005 · Primary buttons disable with no visible reason (systemic)

Routes: `/#/`, `/#/recipes/new`

Three primary actions render greyed-out with the explanation available only to assistive tech:

Button	Route	Reason (only in `aria-label` / not shown)
Cook now	`/#/`	"select one or more meals below"
Shop ingredients	`/#/`	"select meals on this week's menu"
Import recipe	`/#/recipes/new`	(none at all — silently disabled until the URL parses)

The Import case is the worst: type a malformed URL and the button simply never becomes clickable, with no message anywhere on the page. Verified — invalid input keeps `disabled: true`; a valid URL enables it; no hint text appears in either state.

Fix: replace disabled-with-no-reason with either an enabled button that explains on click, or visible helper text under the button. This is one shared pattern worth fixing once.

F-006 · Touch targets are well below 44px throughout the mobile experience

Routes: `/#/`, `/#/shopping`, `/#/history`, `/#/recipes`

Measured at mobile width, on the two most-used screens:

Control	Size	Route
Servings – / + (menu)	**30 × 26**	`/#/`
Item checkbox	**22 × 22**	`/#/`, `/#/shopping`
Remove item ×	**24 × 24**	`/#/shopping`
Servings – / + (list)	36 × 36	`/#/shopping`
Month prev / next ‹ ›	36 × 36	`/#/history`
Filters	65 × 29	`/#/recipes`
"Start with blank recipe"	143 × 20	`/#/recipes/new`

21 controls below 44px tall on the shopping list alone; 14 on the menu. The 22×22 checkbox is the single most important mobile interaction in the app — checking groceries off one-handed in a store — and it is a quarter of the recommended area.

Fix: 44×44 minimum hit area (padding or a `::before` overlay; the visual size need not change).

F-007 · Six routes have no `h1`; page titles are non-semantic `div`s

FIXED — 28 Jul 2026. page titles now render as `<h1>` on the routes that lacked one — verified exactly one h1 on all 9 routes. **Routes:** `/#/`, `/#/recipes`, `/#/recipes/discover`, `/#/recipes/new`, `/#/shopping`, `/#/history`

The visible page title ("My menu", "Recipes", "Add recipe") is a styled `generic`/`div`, not a heading. `/#/shopping` and `/#/history` start their outline at `h2`. Four routes do it correctly (`/#/cooking-now`, `/#/place-order`, `/#/order/kroger`, `/#/order/safeway` all have a proper `h1`), which makes this an inconsistency rather than a convention.

Fix: promote the page title to `h1` on every route.

F-008 · Cook mode's `h1` is "Recipe overview", not the dish being cooked

Route: cook mode (`/#/recipe/:id?cook=1`)

The page's main heading is the generic label "Recipe overview"; the actual dish name sits outside the heading hierarchy. Screen-reader and browser-tab context both lose the one piece of information that identifies the page.

F-009 · Four inputs use placeholders as their only label

Routes: `/#/recipes`, `/#/recipes/discover`, `/#/history`, `/#/recipes/new`

Input	Placeholder
Recipe search	"Search titles, ingredients, steps…"
Discover search	"Search titles, ingredients, steps…"
History filter	"Filter recipes…"
Paste-recipe textarea	"Paste a recipe here — ingredients, instructions…"

Placeholders vanish on focus, are not reliably announced, and fail contrast conventions. Note the app gets this right elsewhere — sign-in has real `<label>`s, and the URL import field has a proper `aria-label`.

F-010 · Sign-in errors are not announced to screen readers

FIXED — 28 Jul 2026. `role="alert"` added to all three sign-in error paragraphs — verified against a real failed sign-in. **Route:** sign-in

Errors render as `<p className="auth-error">` (AuthScreen.tsx:140, and again at :161, :194) with no `role="alert"` and no `aria-live`. A screen-reader user submits the form and hears nothing.

This is inconsistent with the app's own toast system, which does it correctly (ToastContext.tsx:46 — `aria-live="polite"` + `role="status"`).

Fix: add `role="alert"` to the three error paragraphs.

F-011 · Onboarding opens on a Safari-only instruction, shown on every platform

FIXED — 28 Jul 2026. slide gated behind a new `isIosSafari()`; desktop now opens on "Start with your menu" (6 slides, 6 dots). **Route:** onboarding (first launch, before sign-in)

Step 1 of 7 — the very first thing a new user sees — is "Add us to your Home Screen", instructing them to *"In Safari, tap the …, then tap 'Share', 'View More', and 'Add to Home Screen'"*. This was shown verbatim in desktop Chrome at 1280px, where none of those controls exist.

It also means the user is asked to install the app before they know what it does, and before they can even sign in.

Fix: detect iOS Safari before showing this step, and move it later in the sequence (or after first successful use).

F-012 · Ingredient notes fail AA contrast (4.48:1 against a 4.5 requirement)

FIXED — 28 Jul 2026. `#777` -> `#757575`; re-measured at **4.61:1**, passes AA. **Route:** `/#/shopping` · **Selector:** `.shopping-check-note`

`#777777` on white at 12.8px measures **4.48:1** — just under the 4.5:1 threshold for normal-size text. Affects the qualifier notes ("head, florets", "10g", "to toss") — exactly the detail needed while standing in a store.

Fix: darken to `#757575` (4.60:1) or larger. This was the only genuine contrast failure found anywhere in the app.

F-013 · Selection mode gives no visible explanation of what's happening

Route: `/#/recipes?addToPlan=…`

Tapping "+ Add meal" navigates to the recipe list, which silently grows checkboxes and a bottom action bar. The page header still reads "Recipes" and nothing on screen says "pick meals to add to your menu". The intent is encoded only in per-row `aria-label`s ("Add X to your menu selection").

The bottom button does update well — "Add (2) to menu" — which is good feedback once you've figured out the mode.

F-014 · Secondary unit conversions add noise rather than clarity

Route: `/#/shopping`

Every quantity carries a parenthetical conversion, several of which are not useful for shopping:

- `Cheddar cheese - 0.50 cups (0.13 qt)` — quarts for half a cup of cheese
- `Sour cream - 3 tbsp (9 tsp)` — teaspoons for a tablespoon measure
- `Cholula hot sauce - 2 tsp (0.67 tbsp)`
- `Butter - 1 oz (0.06 lb)`

Fix: only show a conversion when it crosses a purchasing threshold (oz → lb above ~16 oz), and drop tsp/tbsp/qt cross-conversions.

F-015 · Quantities use decimals where cooking convention is fractions, inconsistently between views

Routes: `/#/shopping`, `/#/recipe/:id`

The app renders `0.50 cup`, `0.75 cup`, `0.25 cups`, `1.50 lb` — while the human-written instruction text on the same page uses `1/2-inch` and `1/2 cup`. Two conventions on one screen.

Worse, the two halves of the shopping list disagree with each other for the same ingredient:

View	Rendering
Aggregated (by aisle)	`Cream cheese - 0.25 cups`
By recipe	`Cream cheese - 4 tbsp`

Same quantity, two units, one screen. Also inconsistent pluralisation between the views (`0.50 cups` vs `0.50 cup`).

F-016 · Pluralisation bug: "1 pouche"

FIXED — 28 Jul 2026. plural map inverted instead of trimming a trailing "s" — also fixed `bunches` and `boxes`. Verified "1 pouch". **Route:** `/#/shopping`

The aggregated view renders `Beef stock concentrate - 1 pouche`, while the by-recipe view of the same item renders `1 pouch 10g`. A singularisation rule is mangling "pouches" → "pouche".

F-017 · Focus is not moved into the nav drawer when it opens

Route: all

Opening the menu leaves focus on the toggle button; the drawer contents are reached only by tabbing forward. The rest of the drawer implementation is genuinely good — `aria-expanded` toggles correctly, body scroll locks (`overflow: hidden`), Escape closes it, and **focus is correctly returned to the trigger on close** — which makes the missing open-side focus move the one gap.

F-018 · Heading level skips from `h2` to `h4`

Route: `/#/shopping`

Outline runs `h2 "Recipes (# of servings)"` → `h3 aisle groups` → `h2 "By recipe"` → `h4 recipe names`, skipping `h3`.

F-019 · Incomplete ARIA tab pattern

Route: `/#/recipes/new`

The Paste text / Share photo / Link URL tabs set `role="tab"` and `aria-selected` correctly, but there is **no** `aria-controls` **and no** `role="tabpanel"` — so the relationship between tab and content is never exposed. All three tabs also carry `tabIndex=0` rather than the roving tabindex the pattern expects.

P3 findings

F-020 · Long unbroken words overflow their container instead of wrapping or truncating

FIXED — 28 Jul 2026. `overflow-wrap: break-word` plus `min-width: 0` on the flex title row. 571px text in a 343px card now wraps. Title elements compute to `overflow-wrap: normal`, `word-break: normal`,

`text-overflow: clip`, `overflow: visible`. Measured against a 343px phone-width container, a 71-character unbroken title renders **594px wide — overflowing by 251px**. A normal spaced title of similar length wraps correctly to three lines.

Low likelihood (needs a pathological title) but a one-line fix: `overflow-wrap: anywhere` on title elements. *Measured synthetically — not visually confirmed at a real 375px viewport, see limitations.*

F-021 · "edit" link is a 28 × 17px target nested inside the `h1`

Route: `/#/recipe/:id` — both a tiny hit area and heading-name pollution (the `h1` reads "Chicken Parmesan edit"). Its lowercase styling is also inconsistent with every other action label in the app.

F-022 · 12px text on secondary labels

`DAYS COOKED` / `TOTAL SERVINGS` stat labels (`/#/history`), the "In progress" badge (`/#/`), and 12.8px byline/notes elsewhere. Small on a phone.

F-023 · Empty "Recipe not found" state offers no way forward

Renders "Recipe not found." plus a Back button — correct and graceful, but no "Browse recipes" suggestion.

Pass 2 — visual review & iPhone 16 Pro testing

These findings came from the screenshot pass and from testing at **iPhone 16 Pro** dimensions (402 × 874 CSS px, 1206 × 2622 at @3x). Every one of them was invisible to the DOM-measurement pass.

iPhone 16 Pro headline: no horizontal overflow on any of the 9 routes tested, and touch-target violations are identical to 375px (the 402px width changes nothing structurally — both sit in the same sub-520px band). The device pixel ratio could not be set to 3 in this harness, but DPR affects raster sharpness, not layout.

F-024 · Cook mode shows 19 pixels of instructions — the rest is hidden in a nested scroller · P1

Route: cook mode / `/#/cooking-now` · **Category:** layout

The cook-mode panel puts its content in an inner scroll container (`.cook-mode-v2-confirm-scroll`) with a **449px visible height against 916px of content — 467px hidden**. Because the full ingredient list renders above it, the INSTRUCTIONS heading lands at the very bottom edge and only **19px of instruction text is visible**, sliced through the middle of the first line.

The screen whose entire purpose is "read this while you cook" devotes almost all of its space to the ingredients you already bought, and requires scrolling a nested region to read a single step.

Fix: let the panel scroll with the page, or collapse ingredients by default in cook mode so instructions lead.

F-025 · Pages are fixed to viewport height with inner scrollers, so the document never scrolls · P2

Routes: `/#/`, `/#/history`, `/#/cooking-now`, `/#/place-order`, `/#/order/kroger`, `/#/recipes/new`

On six of nine routes the document height is exactly the viewport height (874px) — the page itself cannot scroll, and content scrolls inside nested containers instead.

Worst case is the **"Log a meal" picker on** `/#/history`: `.planner-sheet-body` shows **509px of a 2,979px list — 2,470px hidden**, on a phone with 874px of screen. The 71-recipe list is squeezed into 58% of the display while the document is locked.

Why it matters on iOS specifically: Safari only collapses its URL bar in response to *page* scrolling. A page that never scrolls keeps that chrome permanently, costing another ~60–100px. Nested scrollers also interact badly with momentum scrolling and rubber-banding — overscroll frequently grabs the wrong container.

F-026 · Disabled buttons are styled by opacity alone, so a disabled primary still looks clickable · P2

Routes: `/#/`, `/#/shopping`, `/#/recipes/new`

Disabled state is communicated purely by `opacity: 0.5` (plus `cursor: not-allowed`, which does nothing on touch). Because that preserves each button's base styling, two buttons in the *same* state read as opposite states:

Button	State	Appearance	Effective label contrast
Cook now	disabled	solid green, reads as an active primary	**1.6:1**
Shop ingredients	disabled	nearly invisible against the page	**2.29:1**

Disabled controls are exempt from WCAG contrast minimums, but 1.6:1 is unreadable, and a disabled primary that still looks like the page's main call to action is a false affordance — the user taps and nothing happens, with no explanation (see F-005).

F-027 · The destructive "Clear list" sits beside the primary CTA at equal visual weight · P2

Route: `/#/shopping`

"Place order" (filled green) and "Clear list" (outlined, same size, same row) are immediate neighbours. The irreversible, unconfirmed action from F-001 is one mis-tap away from the primary action, at thumb height on a phone.

Fix: demote "Clear list" to a text button, move it away from the primary CTA, or put it behind an overflow menu — in addition to the confirmation from F-001.

F-028 · Quantities render as unreduced mixed units · P2

Route: `/#/shopping`

Scaled quantities are not normalised before display:

- `Parmesan cheese - 1 cup + 8 tbsp (0.38 qt)` — 8 tbsp is exactly ½ cup; this should read **1½ cups**
- `Olive oil - 1 cup + 1.33 tbsp (0.27 qt)` — hundredths of a tablespoon are meaningless at a shelf

This compounds F-014 and F-015: a single line can carry a decimal, an unreduced mixed unit, *and* an unhelpful parenthetical conversion at once.

F-029 · Meal cards use colour to encode side-vs-main, with no legend · P3

Route: `/#/`

Cards render in two different tints driven by a `side` class — mains get green (`#e8eee8`), sides get blue (`#e6f5f1`). Nothing on screen explains the distinction, and it is carried by colour alone, so it is invisible to colour-blind users and to anyone who hasn't been told.

Fix: add a small "Side" text label, or drop the colour distinction.

F-030 · Recipe list cards spend ~132px of height on one line of text · P3

Route: `/#/recipes`

Each recipe is a full-width card roughly 132px tall containing only its title — about 8 recipes per phone screen, with no image, cook time, servings, or tags to justify the space. Scanning a larger library will mean a lot of scrolling, and the cards carry no information a plain list wouldn't.

Tags — added after the main passes

Full analysis and the per-recipe migration table live in `docs/tag-reconciliation.md`.

F-031 · Tags cannot be edited anywhere in the app · P2

Routes: all · **Category:** missing capability

`tags` appears in only three files across `src/`. `RecipeList.tsx` and `DiscoverPage.tsx` read them for filtering; EditRecipePage.tsx:505 references them exactly once, as `tags: []` when initialising a blank recipe. **There is no UI to add, rename, remove, or merge a tag** — not in the recipe editor, not anywhere.

Tags are written once by the AI import path and are immutable from the user's perspective thereafter. That is the direct cause of F-032: the duplicate tags below cannot be fixed by the person who owns the recipes, so they simply accumulate. Correcting a single typo currently requires direct database access.

Fix: a tag editor in `EditRecipePage`, ideally a combobox that suggests existing tags (the app already has three combobox components) so users pick canonical values rather than typing new variants.

F-032 · Duplicate and unnormalised tags fragment the filter row · P2

Route: `/#/recipes`, `/#/recipes/discover` · **Category:** data integrity

21 distinct tags across 66 recipes, including four pairs that are the same concept stored differently: `crock pot` (13) / `crock-pot` (2) · `mexican` (2) / `Mexican` (1) · `italian` (5) / `Italian-American` (1) · `Greek` as the only capitalised cuisine.

Each variant becomes its own filter chip, because `uniqueTags()` in RecipeList.tsx:43 collects raw strings into a `Set` with no normalisation. Two separator conventions coexist (`crock pot` with a space, `air-fryer` and `meal-prep` with hyphens).

Root cause: api/recipes/parse.js:158 types tags as `z.array(z.string())` — an unconstrained free-text array with no enum, no case handling, and no separator rule. Every AI import can mint a new variant, which is exactly where `Italian-American` and `Mexican` came from. The prompt asks only for "cuisine, dietary", yet the live data also contains method, dish-type and workflow tags, so intent and output have drifted.

Filters are also silently wrong. Under-tagging is the bigger practical problem: "Air Fryer Mozzarella Sticks" and "Chicken (air fried)" carry no `air-fryer` tag, and "Crock Pot Chicken & Broccoli-Rice Freezer Burritos" carries no crock-pot tag despite both saying so in the title. Filtering by `air-fryer` today hides 2 of the 8 air-fryer recipes.

Two further data issues surfaced: **"Test recipe2" is live test data** in the recipe list, and there are two near-duplicate pairs tagged inconsistently (two Chicken Parmesans, two Turkey Chilis).

Verified as working (tested, no defect)

Worth recording so nobody re-investigates these:

- **No horizontal overflow on any route at any width tested** (375 / **402 — iPhone 16 Pro** / 525 / 627 / 1280 / 1920).
- **iPhone 16 Pro (402 × 874) is structurally identical to 375px** — same layout, same target sizes, no new overflow.

Nothing about that device needs a dedicated fix beyond the shared mobile findings (F-006, F-024, F-025).

- **The predicted 519–539px CSS "dead zone" is a non-issue.** The stylesheet mixes `max-width: 520` with `min-width: 540`, which looked like an uncovered band — swept at 525px and every route rendered identically to 375px with no overflow and no target-size change. Hypothesis disproved.
- **Typography is well-executed:** 74-character line length, 1.45 line-height, content capped at 608–800px even on a

1920px viewport.

- **Contrast is otherwise clean** — the only failure is F-012. Checked-off items measure 7.63:1 and correctly use strikethrough *plus* colour, so state is never colour-only.
- **Scroll restoration works** — scrolled to 184px, opened a recipe (correctly snapped to top), pressed Back, restored

to exactly 184px.

- **Deep-link hard refresh works** — session, route, and list state all survived a reload on `/#/shopping`.
- **Toasts are correctly announced** (`aria-live="polite"` + `role="status"`).
- **Servings scaling is correct when base servings exist** (4 → 8 doubled every quantity, with correct pluralisation).
- **Nav drawer**: `aria-expanded`, scroll lock, Escape-to-close, and focus-return-to-trigger all correct.
- **Sign-in error handling**: wrong password returns a clean "Invalid username or password" (not a raw API error) and

preserves both field values; empty submit is caught with "Please enter a username and password."

- **Add-item form**: input clears after submit; item is filed under "Additional items" and survives a list rebuild.
- `aria-label` **quality on interactive controls is excellent throughout** — labels are specific, and several are state-aware (a checkbox flips from "Add X to your menu selection" to "Remove X from add-to-menu selection").
- **Uppercase headings use** `text-transform`, not literal capitals, so they read normally in a screen reader.
- **Recipe-not-found** is handled without a crash or blank screen.

Top 10 quick wins (impact ÷ effort)

#	Fix	Finding	Effort
1	`role="alert"` on the 3 sign-in error paragraphs	F-010	1 line ×3
2	`.shopping-check-note` `#777` → `#757575`	F-012	1 line
3	`overflow-wrap: anywhere` on title elements	F-020	1 line
4	Promote page titles to `h1` on 6 routes	F-007	small
5	Cook mode `h1` → dish name	F-008	small
6	Confirmation (or undo toast) on "Clear list"	F-001	small
7	Visible "replaces your current list" text near Shop ingredients	F-002	small
8	44px hit areas on steppers, checkboxes, ×	F-006	medium
9	Gate the Safari "Add to Home Screen" onboarding step to iOS Safari	F-011	small
10	Disable-with-a-visible-reason pattern for the 3 primary buttons	F-005	medium

Two more that came out of the visual pass and rank alongside the above:

#	Fix	Finding	Effort
11	Collapse ingredients in cook mode so instructions lead (19px visible today)	F-024	small
12	Move "Clear list" away from "Place order"	F-027	1 line

Skipped / untested

Area	Why
Screenshot evidence embedded per finding	The browser tooling returns images to the reviewer but cannot save them to disk; visual findings are described and quantified instead
Visual review of `/#/history`, Discover, and the order pages	The browser pane became unavailable again partway through Pass 2; those routes have measurement coverage only

Area	Why
Kroger & Safeway completion	Stopped at the OAuth/handoff boundary per audit guardrails — no account linking, no real carts
Full breakpoint matrix (381 / 539 / 719 / 721 / 768)	Browser pane did not honour every resize request
Recipe create / edit / delete round-trip	AI-backed import path not exercised to avoid external model calls; blank-recipe editor not walked
Discover add-to-saved flow	Not walked
`prefers-reduced-motion`	Emulation unavailable without DevTools protocol access
200% zoom reflow	Requires real viewport control
Drag-to-reorder on the menu	No drag affordance found in the accessibility tree; unverified whether it exists
Multi-toast stacking	Not triggered

Data integrity note

All work ran against the Neon `ui-audit` branch (`br-odd-haze-amyo1fn0`), verified before any mutation by confirming the `DATABASE_URL` host was `ep-polished-feather-am06mifz-pooler`. **Production data was never connected to.** No source files were modified. The branch auto-expires 2026-08-11.

Test data created during the audit (an `[AUDIT] coffee filters` list item, menu entries, a rebuilt shopping list) was left in place on the disposable branch, and the `Clear list` test removed most of it as a side effect.